Running Your Own AI, On Your Own Hardware

A step-by-step guide to running a private LLM on an 8GB-RAM device

---

Written for people building tech in resource-constrained markets, no expensive GPU, no cloud subscription, no data leaving your machine.

Why This Matters

Most AI tools assume you have fast internet, deep pockets, and don't mind your data going to a US server. That's not the reality for most small businesses across Africa, and it doesn't have to be.

This guide walks you through running a real, usable AI language model entirely on a normal laptop with 8GB of RAM. No internet required after setup, no subscription, and nothing you type ever leaves your device.

If you're non-technical: every section has a plain-language summary before the technical steps. You can follow this guide end to end without ever having used a terminal before.
If you're technical: skip straight to the commands, they're all here too.

Part 1: The Idea in Plain Language

A “language model” (the thing behind ChatGPT, etc.) is normally huge: tens of gigabytes, needs a powerful server to run. But there are smaller, compressed versions of these models that trade a little bit of quality for the ability to run on ordinary hardware.

The three pieces you need:

- **A small model**: a compressed AI model file, small enough to fit in memory
- **A runtime**: the program that actually loads the model and lets you talk to it
- **Your computer**: 8GB RAM is enough, no special graphics card required

That's it. Once it's set up, it works completely offline.

Part 2: What We're Using

This guide uses **llama.cpp**, compiled from source, as the runtime. It's more hands-on than an all-in-one tool like Ollama, but it gives full control over performance, which matters on constrained hardware, and it's the actual setup this guide was built and tested on.

For the model, we're using **Phi-3.1-mini-4k-instruct**, in GGUF format, at the **Q4_K_M** quantization level (a good balance of speed and quality, roughly 2.4GB). It's a 3.8B parameter model, small enough to run comfortably on 8GB RAM.

Model file used in this guide: Phi-3.1-mini-4k-instruct-Q4_K_M.gguf, from bartowski/Phi-3.1-mini-4k-instruct-GGUF on Hugging Face.

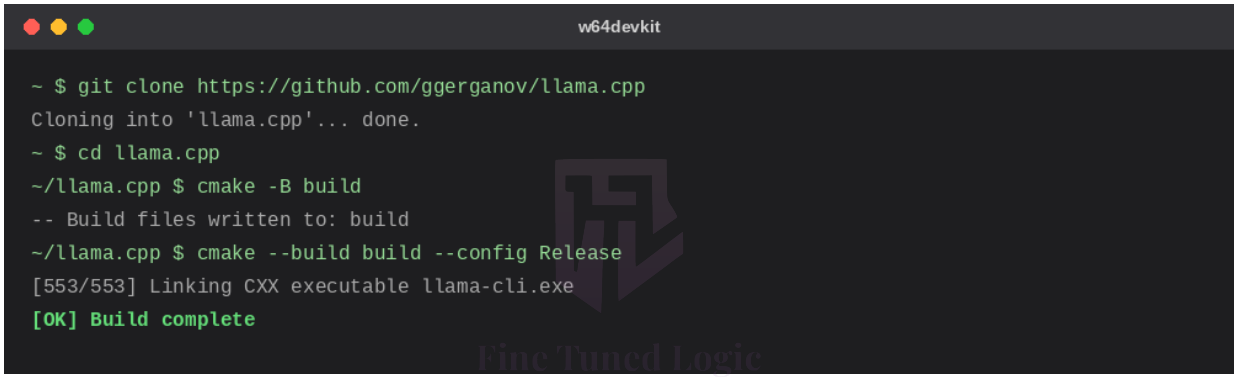
Part 3: Building llama.cpp (Windows)

llama.cpp needs to be compiled from source. On Windows, the simplest path is **w64devkit** (a lightweight build toolchain, no full Visual Studio needed) plus **CMake**.

Step 1: Install the toolchain

- Download w64devkit from github.com/skeeto/w64devkit/releases (the x64 .7z.exe)
- Run it to extract, then launch w64devkit.exe, this opens the terminal you'll use
- Download and install CMake from cmake.org/download, and tick “Add CMake to the system PATH” during install
- Close and reopen the w64devkit terminal so it picks up the new PATH

Step 2: Clone and build



Cloning and building llama.cpp inside w64devkit

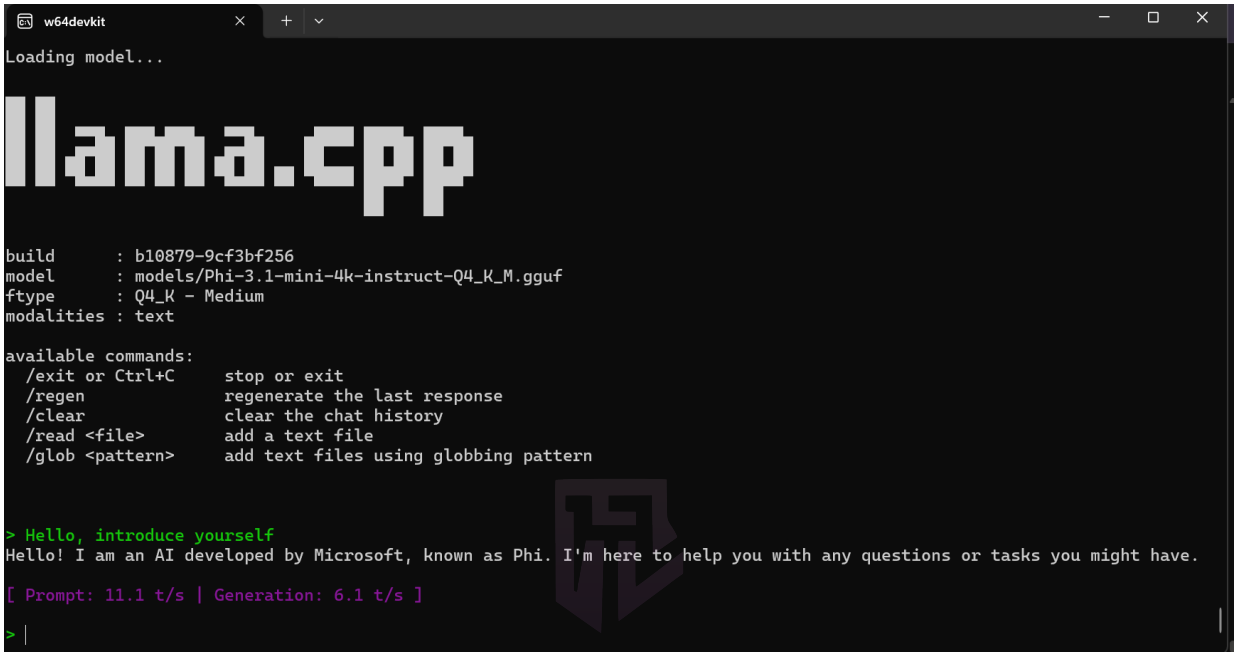
The build compiles 500+ files and can take several minutes depending on your machine. Occasional yellow warning text during the build is normal and can be ignored, only red error text that stops the build needs attention.

Mac/Linux note: the same clone and build steps work with a standard terminal, using your system's existing C++ toolchain (Xcode Command Line Tools on Mac, build-essential on Linux) instead of w64devkit.

Part 4: Getting a Model and Running It

Go to huggingface.co and search for the model (in this guide: “bartowski Phi-3.1-mini-4k-instruct GGUF”). Open “Files and versions” and download the file ending in **Q4_K_M.gguf**. Place it in the models folder inside your llama.cpp directory.

Then run it:



Actual screenshot: the real model responding, running fully offline on 8GB-RAM hardware

That's a real, measured result on ordinary 8GB-RAM hardware: **2,793 MB (~2.8GB) of RAM** used, at **11.1 tokens/sec** processing the prompt and **6.1 tokens/sec** generating the response, with no internet connection required.

Part 5: Using It From Your Phone

The steps above run the model on one machine. To make it reachable from a phone or any other device on the same WiFi network, use **llama-server** instead of llama-cli, and bind it to your network:

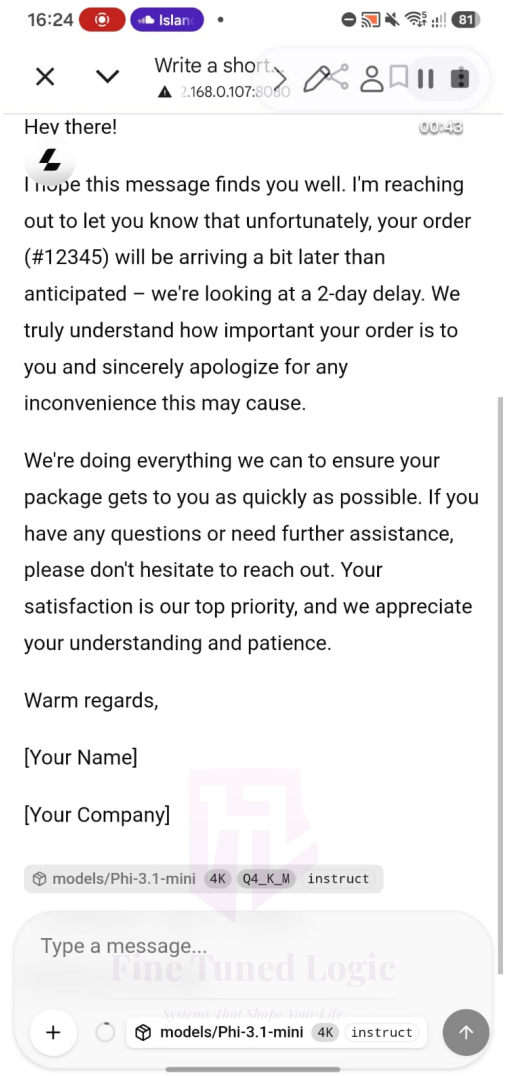
Running the server in network mode, then opening it from a phone browser

To connect from your phone:

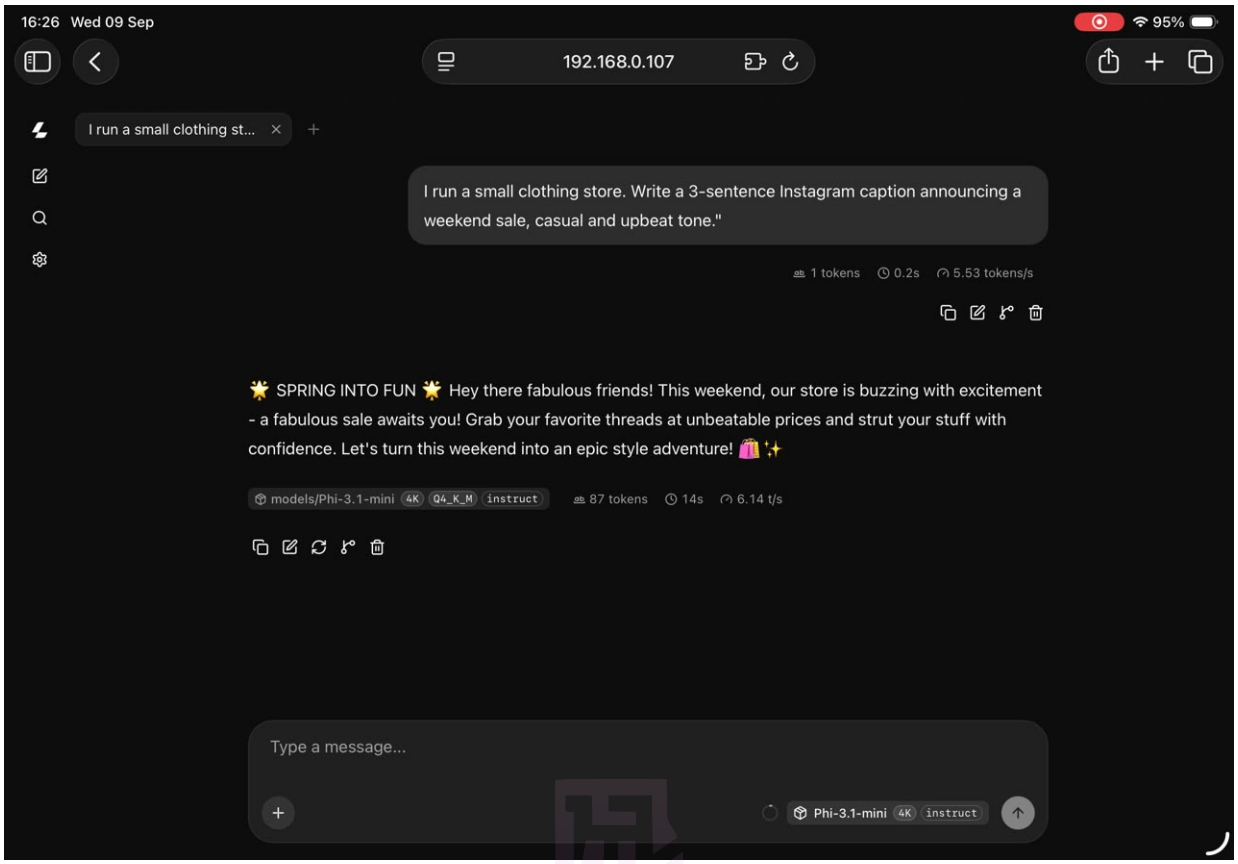
- Find your laptop's local IP address: run **ipconfig** on Windows (or **ifconfig** on Mac/Linux) and look for the IPv4 address under your active WiFi adapter
- Allow the connection through your firewall if prompted
- On your phone, connect to the **same WiFi network**, then open http://[that-ip]:8080 in a browser

This only works while both devices share the same local network and the laptop stays on. That's intentional, nothing goes over the public internet, which keeps the privacy promise of this whole setup intact.

Real test, on real devices: here's this exact setup running on a phone and a tablet, both connected over WiFi to the same laptop, no cloud involved anywhere in the chain.



Mobile: drafting a customer WhatsApp message, 163 tokens at 5.81 tokens/sec



Tablet: drafting an Instagram caption, 87 tokens at 6.14 tokens/sec

Part 6: Making It Actually Useful

The good news: you don't need to add anything extra for a proper chat interface. **llama-server ships its own built-in web chat UI**, visible in the screenshots above, the same familiar chat-window experience as ChatGPT, automatically available at the same address as soon as the server is running. No separate install required.

Connecting it to a task: the real value isn't “a chatbot”, it's the chatbot doing something specific: answering questions about a business's own documents, drafting responses, summarizing records. That connection (sometimes called RAG, Retrieval-Augmented Generation) is the natural next skill to layer on once this basic setup is working.

Part 7: Personalizing It

Notice in the mobile screenshot earlier that the drafted message signed off with **[Your Name]** and **[Your Company]**, placeholders instead of real details. That's fixed with a **system prompt**: persistent instructions the model always follows, even though they're invisible in the chat itself.

- In the chat interface (the same one from the phone/tablet screenshots), open the settings, usually a gear icon in the sidebar
- Find the field called **System Prompt** or **System Message**
- Enter real details, for example: "You are a personal assistant for [Real Name] at [Real Business Name] in [Real City]. When drafting messages, always sign off using these real details instead of placeholders."
- Save it, then try the same kind of prompt again, it should now use the real details automatically

This is per-device, not shared. Each person sets their own system prompt in their own browser on their own phone or laptop, stored locally on that device. Everyone in a group can personalize it individually without affecting anyone else's setup.

A good system prompt can go beyond just a name: tone preferences, business hours, common phrases, response length, anything that would otherwise need repeating in every single prompt. For a setup that should feel the same for every user by default, launch the server with a built-in starting prompt using the **-sys "..."** flag, individual users can still override it in their own browser afterward.

Part 8: Managing the 8GB Constraint

- **It will be slower than ChatGPT.** That's the trade-off for running locally on modest hardware.
- **Close other heavy programs** while running the model to free up RAM.
- **If it's too slow or crashes:** drop to a smaller model, or a more compressed quantization level (e.g. Q4_K_S instead of Q4_K_M).
- **Realistic expectations:** great for drafting, summarizing, and business-specific tasks. Not a match for the largest cloud models on complex reasoning, and for most SME use cases, it doesn't need to be.

Quick Reference: Full Setup

```
# 1. Install w64devkit and CMake (Windows), or your system
#    build tools (Mac/Linux)

# 2. Clone and build
$ git clone https://github.com/ggerganov/llama.cpp
$ cd llama.cpp
$ cmake -B build
$ cmake --build build --config Release

# 3. Download a model (Q4_K_M .gguf) from Hugging Face
#    into the models/ folder

# 4. Run it locally
$ build/bin/llama-cli -m models/your-model.gguf -p "Hello"

# 5. Or run it for the whole network (mobile access)
$ build/bin/llama-server -m models/your-model.gguf \
      --host 0.0.0.0 --port 8080
# then visit http://[your-laptop-ip]:8080 from your phone
```

What's Next

- Connecting it to a business's own documents (private Q&A; over their own data)
- Adding a proper chat interface (Open WebUI) so non-technical staff aren't looking at a terminal or bare browser page
- Testing on the actual hardware a business already owns, to confirm real-world performance

---

This guide is part of ongoing work under Fine Tuned Logic: practical AI and automation solutions built for the hardware realities of the African market.